

CORE PLATFORM FRAMEWORK

OFFICIAL PRODUCT DOCUMENTATION

Build · Run · Recover · Audit

master 64fd08d96392 | source 4c4248a12e69 | 186 requirements



CORE PLATFORM FRAMEWORK

02 배치개발매뉴얼

Spring Batch · Scheduler · Center-Cut · Agent/Runner/Worker ·
Restart/Reprocess/Reconcile

항목	기준
대상	배치 개발자, 배치 운영 개발자, Tech Lead
정본 역할	Batch 개발·실행·복구 정본
기준 Repository	freeangelsun/202412_01_CPF · master
기준 Commit	64fd08d963927860e8d023403dfa276931801ee5
요구사항 정본	CPF_FINAL_TARGET_REQUIREMENTS.md · 186 개
현행화 Source	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (`07_16`, canonical currentization source)

이 문서는 현재 구현의 상태 보고가 아니라, CPF 제품 사용자가 업무를 끝내기 위해 따라야 할 완성 기준 사용 계약을 중심으로 작성한다. 실제 CPF Symbol/Path 는 CURRENT SOURCE, 구현 독립 예시는 REFERENCE 로 구분한다.

문서 읽는 법과 사실 구분

표기	의미	사용 규칙
CURRENT SOURCE	최신 master 에서 직접 확인한 실제 Class/API/Route/Property/SQL/Test/경로	복사·검색 가능한 실제 이름으로 사용한다.
PRODUCT CONTRACT	CPF 가 완료된 제품으로서 제공해야 하는 사용·운영 계약	현재 구현량 때문에 축소하지 않는다.
REFERENCE	사용자가 그대로 응용할 수 있도록 제공하는 완성 기준 코드·설정·절차 예시	실제 CPF 내부 Symbol 인 것처럼 표현하지 않는다.

본문은 기능을 찾는 데서 끝나지 않는다. 적용 가능한 범위에서 선택 기준 → 준비 → 구현/설정 → 실행 → 정상 결과 → 실패/부분 실패 → 복구·대사 → 보안·감사 → Test → 운영 인계 순서로 설명한다.

1. Batch Architecture 와 Primary Engine

Spring Batch 가 Job/Step/JobRepository/ExecutionContext/Restart 의 Primary Engine 이다. CPF 는 Spring Batch 를 대체 하는 별도 실행 Engine 을 만들지 않고 승인/idempotency/fencing/UNKNOWN/운영 원장을 Spring Batch identity 와 연결한다.

● ● ● FLOW · Spring Batch 가 Job/Step/JobRepository/ExecutionContext/Restart 의 P

Scheduler / Operator

→ approved Job Definition + immutable parameter hash

→ JobInstance / JobExecution

→ StepExecution

→ Tasklet | Chunk(Reader→Processor→Writer)

→ Partition / Remote Worker / Center-Cut

→ Business Facade / DB / Message / External

→ Checkpoint / Restart / Reprocess / Reconcile

→ ADM Batch Workbench

2. Job·Step·Tasklet·Chunk 선택

형태	선택 기준	Transaction/Restart 포인트
Tasklet	단일 명령/파일 이동/정리	Tasklet 단위로 idempotency 와 restart 의미 명시
Chunk	대량 item 처리	chunk commit interval + ExecutionContext/checkpoint
Partition	독립 분할 가능한 대량 처리	partition identity/claim/lease/fence
Remote Worker	별도 process/host scale-out	durable correlation, no in-memory fallback
Center-Cut	대상 생성·분배·속도제어·재처리	item/attempt/aggregate 상태와 failed-only reprocess

2.1 REFERENCE Chunk Job

[REFERENCE]

REFERENCE - Spring Batch 표준 구조

● ● ● CODE · REFERENCE - Spring Batch 표준 구조

```
@Bean
Job settlementJob(JobRepository repo, Step settlementStep) {
    return new JobBuilder("settlementJob", repo)
        .start(settlementStep)
        .build();
}

@Bean
Step settlementStep(JobRepository repo,
    PlatformTransactionManager tx,
    ItemReader<SettlementItem> reader,
    ItemProcessor<SettlementItem, SettlementCommand> processor,
    ItemWriter<SettlementCommand> writer) {
    return new StepBuilder("settlementStep", repo)
        .<SettlementItem,SettlementCommand>chunk(500, tx)
        .reader(reader)
        .processor(processor)
        .writer(writer)
        .faultTolerant()
        .retryLimit(3)
        .retry(TransientDependencyException.class)
        .skipLimit(100)
        .skip(BusinessRejectException.class)
        .build();
}
```

3. JobParameter·Metadata·Checkpoint

항목	규칙
JobParameter	재시작 의미에 포함되는 identifying parameter 를 고정
Definition version	승인된 definition/plan checksum 기록
ExecutionContext	Secret/대용량 payload/stdout 전체 저장 금지
Checkpoint	commit 된 범위 이후부터 재개할 수 있는 durable state
TransactionId	원 업무 transactionId/requestId 와 Job/Execution/Step/Partition identity 연결

JobParameter 예

● ● ● CODE · JobParameter 예

```
businessDate=2026-08-08
sourceSnapshotId=SNAP-20260808-01
jobPackVersion=3.2.1
parameterHash=<sha256 canonical parameters>
requestId=BAT-REQ-00001
transactionId=<trusted root or operator-created batch transaction>
```

4. 실제 Batch A→B→C Reference

[CURRENT SOURCE] [cpf-reference/src/main/java/com/cpf/reference/batch/integrated/BatchAbcReferenceFlow.java](#)

현재 Reference 는 Scheduler/Operator→Job→Step/Chunk→Domain A→B→C→DB/Remote 를 연결하고 transactionId/job/execution/step/attempt/fence 를 분리한다. durable FileStore 를 통해 process kill 후 checkpoint/terminal item 을 복원하며 lease/fencing 으로 stale worker 를 차단한다.

CURRENT SOURCE identity/state

● ● ● SOURCE · CURRENT SOURCE identity/state

```
public enum State { RUNNING, RETRYING, SUCCESS, UNKNOWN, FAILED }
public record Identity(String transactionId, String jobId, String executionId,
    String stepId, int attempt, long fenceToken) {}
public record Snapshot(Identity identity, State state, int checkpoint,
    int committed, int remoteEffects, int skipped,
    List<Event> events) {}
```

CURRENT SOURCE Lease/Fence

● ● ● SOURCE · CURRENT SOURCE Lease/Fence

```
public synchronized long acquire(String candidate) {
    if (owner != null && !owner.equals(candidate)) throw new LeaseBusy();
    owner = candidate;
    return fence.incrementAndGet();
}
public synchronized boolean valid(String candidate, long token) {
    return candidate != null && candidate.equals(owner) && fence.get() == token;
}
```

CURRENT SOURCE stale writer/process kill

● ● ● CODE · CURRENT SOURCE stale writer/process killCURRENT SOURCE stale wri

```
if (!lease.valid(owner, id.fenceToken())) {
    events.add(event(id, item.businessKey(), "STALE_FENCE", "lease-lost"));
    return snapshot(id, State.UNKNOWN, events);
}
...
if (killAfter == i) throw new ProcessKilled();
```

5. Reader·Processor·Writer 상세 규칙

Reader

항목	내용
목적	결정적 순서와 stable key 로 대상을 읽는다.
대상 역할	배치 개발자
Owner	Job owner domain + cpf-batch contract
실제/대표 Consumer	Reader
선행 조건	schema/index/sort/key 준비
입력·기본값·범위	chunk size, fetch size, retry/skip cap
정상 흐름	결정적 순서와 stable key 로 대상을 읽는다.
정상 판정	commit count/expected business aggregate 일치
오류·경계·부분 실패	cursor/checkpoint 와 source snapshot drift
복구·대사·Rollback	checkpoint 기반 restart 또는 failed-only reprocess
권한·보안·감사	민감 payload masking, 운영 재처리 승인
Log·Metric·Trace	item/chunk latency/count/error
Test	normal/boundary/restart/duplicate
운영 인계	ADM execution/step/checkpoint 확인

Processor

항목	내용
목적	업무 검증·변환. 외부 side effect 최소화.
대상 역할	배치 개발자
Owner	Job owner domain + cpf-batch contract
실제/대표 Consumer	Processor
선행 조건	schema/index/sort/key 준비
입력·기본값·범위	chunk size, fetch size, retry/skip cap
정상 흐름	업무 검증·변환. 외부 side effect 최소화.
정상 판정	commit count/expected business aggregate 일치
오류·경계·부분 실패	skippable business rejection vs retryable technical failure
복구·대사·Rollback	checkpoint 기반 restart 또는 failed-only reprocess
권한·보안·감사	민감 payload masking, 운영 재처리 승인
Log·Metric·Trace	item/chunk latency/count/error
Test	normal/boundary/restart/duplicate

항목	내용
운영 인계	ADM execution/step/checkpoint 확인

Writer

항목	내용
목적	commit boundary 안에서 DB side effect 를 묶는다.
대상 역할	배치 개발자
Owner	Job owner domain + cpf-batch contract
실제/대표 Consumer	Writer
선행 조건	schema/index/sort/key 준비
입력 · 기본값 · 범위	chunk size, fetch size, retry/skip cap
정상 흐름	commit boundary 안에서 DB side effect 를 묶는다.
정상 판정	commit count/expected business aggregate 일치
오류 · 경계 · 부분 실패	partial write, optimistic lock, duplicate
복구 · 대사 · Rollback	checkpoint 기반 restart 또는 failed-only reprocess
권한 · 보안 · 감사	민감 payload masking, 운영 재처리 승인
Log · Metric · Trace	item/chunk latency/count/error
Test	normal/boundary/restart/duplicate
운영 인계	ADM execution/step/checkpoint 확인

6. Stop·Restart·Abandon

조치	언제	판정/주의
Stop	안전 지점에서 추가 item 처리를 중단	in-flight chunk commit/rollback 상태 확인
Restart	같은 JobInstance 의 실패/중지 execution 을 checkpoint 부터	parameter identity/definition version 불변
Abandon	재시작하지 않을 execution 을 운영 확정	사유/권한/audit, 업무 미처리 범위 별도 처리
New Run	새 업무 기준/parameter	기존 execution 과 lineage 연결

7. Retry·Skip·Rollback 설계

분류	예	행동
Retryable	일시 DB connection, bounded remote 503	bounded retry + backoff; idempotency 전제
Skippable	업무 validation reject	skip reason/record/audit, 한도 초과 시 step fail
Fatal	schema mismatch, auth failure	재시도 없이 fail, retry 금지
Unknown	remote send 뒤 response loss	재호출 금지, result query/reconcile

8. Partition·Remote Worker·Lease/Fencing

Scale-out 안전 모델

● ● ● FLOW · Scale-out 안전 모델

```
partitionKey = businessDate + ':' + shardNo
claim(owner, leaseUntil, fencingToken)
→ worker reads assigned range
→ every state-changing write validates current fencing token
→ heartbeat/renew
→ lease expiry
→ new owner gets higher fencing token
→ old worker late write rejected
```

9. Scheduler·Misfire

상황	처리
정상 trigger	calendar/timezone/parameter snapshot 고정
scheduler leader loss	cluster claim + durable trigger state 로 takeover
misfire	SKIP/FIRE_ONCE/RESCHEDULE 등 승인 정책
trigger 성공/Job start 응답 유실	outbox/state machine 으로 reserve→start→bind 대사
duplicate trigger	canonical request hash/idempotency conflict 검사

10. Center-Cut

Target Generation

immutable parameter snapshot 으로 target/item 생성

Claim

item claim/lease/fence/chunk assignment

Global Rate

domain/target TPS/RPS, concurrency, backpressure, pause/drain

Aggregate

성공/실패/UNKNOWN/skip 합계와 업무 금액/건수 대사

Reprocess

failed-only/selected/range, prior result 보존, approval

UNKNOWN

item/attempt 결과 조회 후 reconcile/compensation/manual confirmation

11. Agent·Runner·Worker·Artifact

항목	내용
목적	승인된 Job Pack/Script 를 신뢰 가능한 process 에서 실행
대상 역할	배치 운영 개발자/SRE
Owner	cpf-batch
실제/대표 Consumer	Host Agent/Runner/Worker
선행 조건	artifact manifest/signature, capability/zone, service account
입력·기본값·범위	timeout/output budget/concurrency/drain
정상 흐름	claim→artifact verify→process spawn→heartbeat→collect→terminal state
정상 판정	exit/result + owner execution state + checksum 일치
오류·경계·부분 실패	process tree leak/kill/host loss/output overflow
복구·대사·Rollback	drain/takeover/restart/reconcile
권한·보안·감사	approved artifact only, least privilege, secret redaction
Log·Metric·Trace	agent/worker heartbeat, process, stdout summary metric
Test	process kill, ghost worker, stale fence
운영 인계	ADM batch-runtime/agents/workers

12. Dry Run·Preview·승인

REFERENCE - Preview/Approval binding

● ● ● CODE · REFERENCE - Preview/Approval bindingREFERENCE - Preview/Approval

```
Preview = {  
  parameterHash,  
  targetCount,  
  estimatedRows,  
  estimatedAmount,  
  partitions,  
  expectedRuntimeClass,  
  riskLevel,  
  sampleKeys(masked)  
}
```

Approval binds to jobDefinitionVersion + parameterHash + targetSnapshotHash.

13. UNKNOWN_RESULT·Reprocess·Reconcile

배치 item 결과 불명 판단

● ● ● FLOW · 배치 item 결과 불명 판단

Remote side effect confirmed? — yes → local state repair / mark reconciled

|

└ no

└ confirmed not processed → safe retry/reprocess

└ still unknown → manual review / compensation policy

Never: UNKNOWN → blind rerun

14. ADM 운영 연계

Route	운영 목적
/batch	Job 등록/실행/관계
/batch-overview	Job/Schedule/Instance/Worker/Lock 요약
/batch-runtime	Runtime topology/command
/batch-scheduler	schedule simulation/enable/disable/run once
/batch-worker-pools	worker pool runtime command
/batch-center-cut	job/result/target/parameter/reprocess/reconcile
/batch-agents	host agent
/batch-job-packs	definition validate/save/transition
/batch-executions	execution detail/retry/stop
/batch-deployment	deployment plan/rollback
/batch-recovery	ghost/unknown/recovery
/batch-leases	lock/release
/batch-alerts	unknown/DLQ/outbox
/batch-audit	operation/audit/evidence

15. 실행형 EDU

15.1 Chunk commit/restart

단계	실행/판정
준비	중간 kill 후 checkpoint 부터 재개를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.2 Retry/Skip

단계	실행/판정
준비	transient retry 와 business skip 분리를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.3 Partition

단계	실행/판정
준비	2+ worker claim/fence 를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행

단계	실행/판정
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.4 Remote worker loss

단계	실행/판정
준비	heartbeat loss→lease expiry→takeover 를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.5 Scheduler misfire

단계	실행/판정
준비	정책별 trigger outcome 를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.6 Center-Cut failed-only

단계	실행/판정
준비	실패 item 만 새 reprocess lineage 를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger

단계	실행/판정
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.7 UNKNOWN item

단계	실행/판정
준비	remote effect/result 대사를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.8 Job Pack trust

단계	실행/판정
준비	hash/signature 불일치 fail 를 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.9 Agent process kill

단계	실행/판정
준비	process tree cleanup/restart 를 재현할 격리 데이터/parameter snapshot 준비

단계	실행/판정
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

15.10 Batch→Domain lineage

단계	실행/판정
준비	transactionId + execution identity 를 ADM 에서 연결을 재현할 격리 데이터/parameter snapshot 준비
실행	Job 을 시작하고 execution/job/step identity 기록
Fault	지정 failure 를 trigger
관측	Spring Batch metadata + CPF operation/unknown/lease state 확인
복구	restart/reprocess/reconcile/takeover 수행
완료	중복 business side effect 0, expected count/amount/checkpoint 일치
ADM	관련 batch route 에서 lineage/audit 확인

Source Trace

구분	Repository 경로	확인 포인트
Batch Reference	cpf-reference/src/main/java/com/cpf/reference/batch/integrated/BatchAbcReferenceFlow.java	checkpoint/retry/skip/fence/kill/UNKNOWN
Batch Modules	settings.gradle	contract/runtime/control/scheduler/worker/center-cut/agent/testkit
ADM Routes	cpf-admin/frontend/src/app/routes.ts	batch-* route/operation mapping
Requirement	cpf-docs/governance/CPF_FINAL_TARGET_REQUIREMENTS.md	BAT-*/CENTER-*

16. Batch 업무 유형별 구현 Recipe

16.1 DB→DB 정산

구분	내용
Flow	JdbcPagingItemReader→Processor→JdbcBatchItemWriter
Control	chunk 500, optimistic lock
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	DB deadlock/kill
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.2 File→DB

구분	내용
Flow	stream reader→validation→writer
Control	line checkpoint/checksum
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	partial file/bad row/disk
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.3 DB→External API

구분	내용
Flow	reader→remote processor→result writer
Control	idempotency+unknown ledger
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	response loss
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.4 Message-driven batch

구분	내용
Flow	broker intake→partition→domain
Control	stable correlation/inbox
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	rebalance/duplicate
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.5 Remote Partition

구분	내용
Flow	manager→durable broker→workers
Control	partition/fence/backpressure
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	worker loss
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.6 Center-Cut

구분	내용
Flow	target generator→item claim→worker
Control	global rate+failed-only
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	stale lease/partial
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.7 Shell/Process

구분	내용
Flow	approved artifact executor
Control	timeout/output/process tree
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	host kill
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

16.8 File Watch

구분	내용
Flow	watch→immutable snapshot→job launch
Control	duplicate event/idempotency
JobParameter	businessDate/sourceSnapshot/definitionVersion/requestId
Checkpoint	마지막 commit/partition/item identity 를 durable 하게 기록
Fault	late/duplicate event
Recovery	Spring Batch restart 또는 selected/failed-only reprocess; UNKNOWN 이면 결과 조회 우선
Reconcile	source count/hash ↔ processed/success/fail/unknown ↔ target effect
ADM	execution/step/worker/lease/unknown/audit

17. Batch EDU 상세

17.1 Chunk kill/restart

목표: checkpoint 이후부터 재개, committed item 중복 0

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 3 개 chunk 중 2 번째 commit 후 process kill 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	checkpoint 이후부터 재개, committed item 중복 0; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-01 --business-date 2026-08-08
inject-fault "3 개 chunk 중 2 번째 commit 후 process kill"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.2 Retry/Skip

목표: timeout 은 retry, reject 는 skip reason

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 1 회 timeout + validation reject 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	timeout 은 retry, reject 는 skip reason; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-02 --business-date 2026-08-08
inject-fault "1 회 timeout + validation reject"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.3 Partition competition

목표: 1 owner only, stale fence reject

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 2 worker same partition 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	1 owner only, stale fence reject; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-03 --business-date 2026-08-08
inject-fault "2 worker same partition"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.4 Worker loss

목표: new worker higher fence takeover

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 heartbeat stop/lease expiry 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	new worker higher fence takeover; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-04 --business-date 2026-08-08
inject-fault "heartbeat stop/lease expiry"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.5 Scheduler misfire

목표: 정책에 따른 skip/fire-once/reschedule

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 trigger 지연 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	정책에 따른 skip/fire-once/reschedule; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-05 --business-date 2026-08-08
inject-fault "trigger 지연"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.6 Remote response loss

목표: UNKNOWN item, blind retry 금지

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 remote effect success/response lost 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	UNKNOWN item, blind retry 금지; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-06 --business-date 2026-08-08
inject-fault "remote effect success/response lost"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.7 Center-Cut reprocess

목표: 새 reprocess execution 이 원 item lineage 참조

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 실패 item 3 건 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	새 reprocess execution 이 원 item lineage 참조; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-07 --business-date 2026-08-08
inject-fault "실패 item 3 건"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.8 Agent kill

목표: process tree 정리 + operation 상태 확정/unknown

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 child process 포함 kill 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	process tree 정리 + operation 상태 확정/unknown; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-08 --business-date 2026-08-08
inject-fault "child process 포함 kill"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.9 Artifact tamper

목표: 실행 전 fail-closed

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 job pack hash mismatch 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	실행 전 fail-closed; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-09 --business-date 2026-08-08
inject-fault "job pack hash mismatch"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

17.10 Batch→ADM lineage

목표: job/execution/step/partition/worker/attempt tree

단계	수행
Given	격리된 job parameter 와 deterministic data 준비
When	Job launch 후 transactionId 로 검색 주입
Observe	Spring Batch metadata + CPF execution/lease/unknown state + transactionId 수집
Recover	restart/reprocess/reconcile/takeover 중 계약 경로 수행
Then	job/execution/step/partition/worker/attempt tree; 총 건수/금액/checkpoint/side effect 대사
Audit	operator/reason/approval 가 필요한 조치는 operation audit 확인

운영 Harness 예

● ● ● TERMINAL · REFERENCE 실행 순서

```
# REFERENCE 실행 순서
run job --scenario BAT-LAB-10 --business-date 2026-08-08
inject-fault "transactionId 로 검색"
query execution --include-steps --include-lineage
recover --mode <restart|reprocess|reconcile|takeover>
verify --counts --amounts --duplicates --unknowns
```

19. 실행형 Batch 개발 워크북

이 장의 목적

Spring Batch Job 을 코드로 만들고 실행·실패·Stop·Restart·Reprocess·Reconcile 까지 따라갑니다. 표만 읽고 끝내지 않고 Source 와 명령·상태를 함께 제공합니다.

19.1 Job/Step/Chunk 전체 Reference

● ● ● SOURCE · PaymentSettlementJobConfig.java

```
@Configuration
@EnableBatchProcessing
class PaymentSettlementJobConfig {
    @Bean
    Job paymentSettlementJob(JobRepository repo, Step settleStep) {
        return new JobBuilder("paymentSettlement", repo)
            .start(settleStep)
            .build();
    }

    @Bean
    Step settleStep(JobRepository repo, PlatformTransactionManager tx,
        ItemReader<PaymentRow> reader,
        ItemProcessor<PaymentRow, Settlement> processor,
        ItemWriter<Settlement> writer) {
        return new StepBuilder("settleStep", repo)
            .<PaymentRow, Settlement>chunk(200, tx)
            .reader(reader)
            .processor(processor)
            .writer(writer)
            .faultTolerant()
            .retry(TransientRemoteException.class).retryLimit(2)
            .skip(InvalidBusinessRowException.class).skipLimit(100)
            .build();
    }
}
```

● ● ● SOURCE · JdbcPagingItemReader 구성

```
@Bean
JdbcPagingItemReader<PaymentRow> paymentReader(DataSource ds) {
    return new JdbcPagingItemReaderBuilder<PaymentRow>()
        .name("paymentReader")
        .dataSource(ds)
        .selectClause("SELECT payment_id, amount, status")
        .fromClause("FROM pay_payment")
        .whereClause("WHERE status = :status")
        .sortKeys(Map.of("payment_id", Order.ASCENDING))
        .parameterValues(Map.of("status", "READY"))
        .pageSize(200)
        .rowMapper(paymentRowMapper())
        .build();
}
```

● ● ● CONFIG · application.yml

● ● ● SOURCE · PaymentSettlementJobConfig.java

```
spring:
  batch:
    job:
      enabled: false
cpf:
  batch:
    worker:
      concurrency: 4
    lease:
      duration: 30s
      heartbeat: 10s
    chunk:
      default-size: 200
```

● ● ● TERMINAL · Job 실행

```
./gradlew :cpf-reference:bootRun --args='--spring.batch.job.name=paymentSettlement runDate=2026-08-09'
```

● ● ● OUTPUT · 정상

```
Job=paymentSettlement
Step=settleStep
read=10000 write=9988 skip=12 retry=3
BatchStatus=COMPLETED
ExitStatus=COMPLETED
```

19.2 Metadata/Checkpoint 를 SQL 로 확인

● ● ● SQL · Spring Batch metadata 확인

```
SELECT JOB_EXECUTION_ID, STATUS, START_TIME, END_TIME, EXIT_CODE
FROM BATCH_JOB_EXECUTION
ORDER BY JOB_EXECUTION_ID DESC;

SELECT STEP_EXECUTION_ID, STEP_NAME, STATUS, READ_COUNT, WRITE_COUNT,
       COMMIT_COUNT, ROLLBACK_COUNT, READ_SKIP_COUNT, PROCESS_SKIP_COUNT, WRITE_SKIP_COUNT
FROM BATCH_STEP_EXECUTION
WHERE JOB_EXECUTION_ID = ?;
```

19.3 CURRENT SOURCE - Batch A→B→C Reference

CURRENT SOURCE ·

`cpf-reference/src/main/java/com/cpf/reference/batch/integrated/BatchAbcReferenceFlow.java` . transactionId 와 job/execution/step/attempt/fence identity 를 분리하고 checkpoint/restart, retry/skip, process kill, UNKNOWN/reconcile 을 검증하도록 작성돼 있습니다.

● ● ● SOURCE · Step.chunk 핵심

```
public Snapshot chunk(Identity id, List<Item> items, String owner, Lease lease) {
    int i = store.checkpoint();
    for (; i < items.size(); i++) {
        Item item = items.get(i);
        if (store.terminal(item.businessKey())) {
            advanceCheckpoint(i, items.size());
            continue;
        }
        if (!lease.valid(owner, id.fenceToken()))
            return snapshot(id, State.UNKNOWN, events);
        a.handle(id, item, events);
        advanceCheckpoint(i, items.size());
        if (killAfter == i) throw new ProcessKilled();
    }
    return snapshot(id, State.SUCCESS, events);
}
```

● ● ● SOURCE · Reconciler

```
public void reconcile(String businessKey) {
    if (!store.committed(businessKey)) remote.compensate(businessKey);
}
```

19.4 Stop / Restart / Abandon / Reprocess

상태	운영 판단	허용 조치	금지
STARTED/RUNNING	worker/heartbeat 확인	STOP 요청 후 checkpoint 대기	process kill 을 정상 stop 으로 간주
STOPPED	restartable job 인지 확인	같은 identifying parameter 로 restart	새 Job 으로 중복 생성
FAILED	commit/side effect 경계 확인	restart 또는 failed-only reprocess	원인 확인 없는 전체 replay
UNKNOWN	remote/DB side effect 불명	reconcile 후 terminal 결정	blind retry
ABANDONED	재시작 금지 명시	새 승인된 reprocess job	원 Job 재시작

● ● ● TERMINAL · Reference fault test

```
./gradlew :cpf-reference:test --tests "**BatchAbc**"
```

● ● ● CODE · process kill 이후

```
PROCESS_KILLED
state=UNKNOWN
checkpoint=<last committed boundary>
restartRequired=true
remoteEffects=<reconcile target>
```

19.5 Partition/Remote Worker Lease-Fencing

● ● ● FLOW · Multi-instance claim

```
READY → CLAIMED(owner=A,fence=41) → RUNNING
    | heartbeat lost
    └─ lease expires
READY → CLAIMED(owner=B,fence=42) → RUNNING
stale writer A(fence=41) → REJECTED
```

● ● ● FLOW · Multi-instance claim

● ● ● SQL · Worker takeover 판정 예

```
SELECT item_id, owner_id, fence_token, lease_until, state
FROM cpf_batch_work_item
WHERE job_execution_id = ?
AND state IN ('CLAIMED','RUNNING')
ORDER BY lease_until;
```

19.6 Center-Cut Dry Run 과 Reprocess

● ● ● CODE · center-cut job pack

```
jobCode: PAYMENT_EOD
version: 7
target:
  queryId: payment.ready
partition:
  count: 16
chunk:
  size: 200
commit:
  policy: PER_CHUNK
dryRun:
  previewCount: true
reconciliation:
  key: paymentId
  expectedState: SETTLED
```

● ● ● CODE · EXPECTED PREVIEW

```
targetCount=284120
partitions=16
estimatedChunks=1421
invalidCount=0
approvalRequired=true
```

19.7 Batch 운영 인계 Gate

● ● ● TERMINAL · 인계 검증

```
./gradlew :cpf-reference:test --tests "**BatchAbc*"
./gradlew :cpf-batch:test
```

● ● ● CODE · EXPECTED HANDOFF RECORD

```
job/version registered : PASS|FAIL
restartability : PASS|FAIL
process-kill recovery : UNVERIFIED unless executed
multi-instance fencing : UNVERIFIED unless executed
reconciliation count/hash : <evidence>
```

문서 기준 및 검증 범위

항목	기준
Repository / Branch	freeangelsun/202412_01_CPF / master
Work start / latest reviewed HEAD	64fd08d963927860e8d023403dfa276931801ee5 (07_17)
Latest product source change basis	4c4248a12e699c07f9f5fb11fbb33b97ca04077d (07_16 canonical currentization source)
Canonical Requirement	186 개
CURRENT SOURCE	실제 Repository 에서 확인한 Symbol/경로
PRODUCT CONTRACT	현재 구현량과 분리한 제품 목표 계약
REFERENCE	복사·응용 가능한 완성 기준 예시; CURRENT SOURCE 아님
Runtime qualification	문서 세션에서 직접 실행하지 않은 DB/Browser/Multi-instance/Fault 는 미검증